目录

Geometry Template

```
#include<bits/stdc++.h>

#define ld long double

template<typename T>
void Clear(T&x){T y;x.swap(y);}
template<typename... Args>
std::istream& InPut(Args&...x){return(std::cin>>...>>x);}
template<typename... Args>
std::ostream& OutPut(const Args&...x){return(std::cout<<...<<x);}
template<typename...Args>
std::ostream& ErrPut(const Args&...x){return(std::cerr<<...<<x);}
void Flush(){std::cout.flush();}

const ld Eps=1e-6;
const ld Pi=acos(-1.0);

bool Gt(const ld&a,const ld&b){return a-b>Eps;}
bool Ge(const ld&a,const ld&b){return a-b>=-Eps;}
bool Lt(const ld&a,const ld&b){return a-b<-Eps;}
bool Le(const ld&a,const ld&b){return a-b<=Eps;}
bool Eq(const ld&a,const ld&b){return std::fabs(a-b)<=Eps;}
ld Acos(ld t){
    assert(!std::isnan(t));
    return std::acos(std::max<ld>(std::min<ld>(t,1.0),-1.0));
}

struct Points2{
    ld x,y;
    Points2(){}
    Points2(const ld&_x,const ld&_y):x(_x),y(_y){}
    friend std::istream& operator>>(std::istream&in,Points2&p){
        in>>p.x>>p.y;
        return in;
    }
    friend std::ostream& operator<<(std::ostream&out,const Points2&p){
        out<<'('<<p.x<<','<<p.y<<')';
        return out;
    }
    ld abs()const{return std::sqrt(x*x+y*y);}
    ld abs2()const{return x*x+y*y;}
    Points2 operator+(const Points2&p)const{return {x+p.x,y+p.y};}
    Points2 operator-(const Points2&p)const{return {x-p.x,y-p.y};}
    Points2 operator+()const{return{+x,+y};}
    Points2 operator-()const{return{-x,-y};}
    Points2 operator*(const ld&p)const{return {x*p,y*p};}
    Points2 operator/(const ld&p)const{return {x/p,y/p};}
```

```
    bool operator==(const Points2&p)const{return Eq(x,p.x)&&Eq(y,p.y);}
};
ld multi(const Points2&a,const Points2&b){return a.x*b.x+a.y*b.y;}
ld cross(const Points2&a,const Points2&b){return a.x*b.y-b.x*a.y;}
Points2 intersection(const Points2&p1, const Points2&dir1, const Points2&p2, const Points2&dir2){
    double t=cross((p2-p1),dir2)/cross(dir1,dir2);
    return p1+dir1*t;
}

void Andrew(std::vector<Points2> Now,std::vector<Points2>&Ans,bool flag=false){
    if(!flag){
        static std::function<bool(const Points2&,const Points2&)>Cmp=[](const Points2&a,const Points2&b){
            return a.x==b.x?a.y<b.y:a.x<b.x;
        };
        std::sort(Now.begin(),Now.end(),Cmp);
    }
    int len=Now.size();
    int tmp,top=0;Clear(Ans);
    tmp=top=1;Ans.push_back(Now[0]);
    for(int i=1;i<len;i++){
        while(top>tmp&&Le(cross(Ans[top-1]-Ans[top-2],Now[i]-Ans[top-2]),0))
            Ans.pop_back(),top--;
        Ans.push_back(Now[i]);top++;
    }
    tmp=top;
    for(int i=len-2;i>=0;i--){
        while(top>tmp&&Le(cross(Ans[top-1]-Ans[top-2],Now[i]-Ans[top-2]),0))
            Ans.pop_back(),top--;
        Ans.push_back(Now[i]);top++;
    }
}

ld angle(const Points2&a){//求极角
    return atan2(a.y,a.x);
}
ld cosIncAngle(const Points2&a,const Points2&b){
    return multi(a,b)/a.abs()/b.abs();
}
ld incAngle(const Points2&a,const Points2&b){//求夹角
    return Acos(cosIncAngle(a,b));
}
//Andrew 到这个是无缝连接的，要求凸包的斜率是递减的，得到的结果有可能共线，且头接尾
std::vector<Points2>MinkowskiSum(std::vector<Points2>a,std::vector<Points2>b){
    std::vector<Points2>c{a[0]+b[0]};
    for (unsigned i=0;i+1<a.size();++i)a[i]=a[i+1]-a[i];
    for (unsigned i=0;i+1<b.size();++i)b[i]=b[i+1]-b[i];
    a.pop_back(),b.pop_back();
    c.resize(a.size()+b.size()+1);
    std::merge(a.begin(),a.end(),b.begin(),b.end(),c.begin()+1,
        [](const Points2&a,const Points2&b){return Gt(cross(a,b),0);});
    for (unsigned i=1;i<c.size();++i)c[i]=c[i]+c[i-1];
    return c;
}
```

```
struct Lines{//向量左侧
    Points2 a,b;ld ang;
    Lines(Points2 _a=Points2(),Points2 _b=Points2()):a(_a),b(_b){
        ang=angle(b-a);
    }
    Lines(ld A,ld B,ld C){//Ax+By+C>=0
        if(Eq(A,0)){
            a=Points2(0,-C/B),b=Points2(1,-C/B);
            if(Lt(B,0))std::swap(a,b);
        }
        else if(Eq(B,0)){
            a=Points2(-C/A,0),b=Points2(-C/A,1);
            if(Gt(A,0))std::swap(a,b);
        }
        else{
            a=Points2(0,-C/B),b=Points2(1,(-C-A)/B);
            if(Lt(B,0))std::swap(a,b);
        }
        ang=angle(b-a);
    }
};
Points2 intersection(const Lines&a,const Lines&b){
    return intersection(a.a,a.b-a.a,b.a,b.b-b.a);
}
bool onRight(const Lines&l,const Points2&p){//也可能在线上
    return Le(cross(l.b-l.a,p-l.a),0);
}

int getHpi(std::vector<Lines>Arr,std::deque<Lines>&Ans){//请添加边界，如果访问到边界，一定无解
    static std::function<bool(const Lines&,const Lines&)>Cmp1=[](const Lines&a,const Lines&b){
        return Eq(a.ang,b.ang)?onRight(a,b.a):Lt(a.ang,b.ang);
    };
    static std::function<bool(const Lines&,const Lines&)>Cmp2=[](const Lines&a,const Lines&b){
        return Eq(a.ang,b.ang);
    };
    std::sort(Arr.begin(),Arr.end(),Cmp1);Clear(Ans);
    Arr.erase(unique(Arr.begin(),Arr.end(),Cmp2),Arr.end());

    int siz=2;
    Ans.push_back(Arr[0]);Ans.push_back(Arr[1]);
    for(unsigned i=2;i<Arr.size();i++){
        while(siz>=2&&onRight(Arr[i],intersection(Ans[siz-1],Ans[siz-2])))Ans.pop_back(),siz--;
        while(siz>=2&&onRight(Arr[i],intersection(Ans[0],Ans[1])))Ans.pop_front(),siz--;
        Ans.push_back(Arr[i]);siz++;
    }
    while(siz>=2&&onRight(Ans[0],intersection(Ans[siz-1],Ans[siz-2])))Ans.pop_back(),siz--;
    if(siz<=2)return -1;//无解
    return 0;//请自行检查边界
}

struct Points3{
    ld x,y,z;
    Points3(){}
```

```
    Points3(const ld&_x,const ld&_y,const ld&_z):x(_x),y(_y),z(_z){}
    friend std::istream& operator>>(std::istream&in,Points3&p){
        in>>p.x>>p.y>>p.z;
        return in;
    }
    friend std::ostream& operator<<(std::ostream&out,const Points3&p){
        out<<'('<<p.x<<','<<p.y<<','<<p.z<<')';
        return out;
    }
    ld abs()const{return std::sqrt(x*x+y*y+z*z);}
    ld abs2()const{return x*x+y*y+z*z;}
    Points3 operator+(const Points3&p)const{return {x+p.x,y+p.y,z+p.z};}
    Points3 operator-(const Points3&p)const{return {x-p.x,y-p.y,z-p.z};}
    Points3 operator+()const{return{+x,+y,+z};}
    Points3 operator-()const{return{-x,-y,-z};}
    Points3 operator*(const ld&p)const{return {x*p,y*p,z*p};}
    Points3 operator/(const ld&p)const{return {x/p,y/p,z/p};}
    bool operator==(const Points3&p)const{return Eq(x,p.x)&&Eq(y,p.y)&&Eq(z,p.z);}
};
Points3 cross(const Points3&a,const Points3&b){
    return {a.y*b.z-b.y*a.z,a.z*b.x-b.z*a.x,a.x*b.y-b.x*a.y};
}
ld multi(const Points3&a,const Points3&b){return a.x*b.x+a.y*b.y+a.z*b.z;}
Points3 project(const Points3&p,const Points3&a,const Points3&b){
    Points3 v=cross(a,b);
    return v-v*multi(v,p)/v.abs2();
}
ld cosIncAngle(const Points3&a,const Points3&b){return multi(a,b)/a.abs()/b.abs();}
ld incAngle(const Points3&a,const Points3&b){//求夹角
    return Acos(cosIncAngle(a,b));
}

typedef Points2 Points;

void Main(int Case){
    Points a=Points(10000,10000),b=Points(-10000,10000),c=Points(-10000,-10000),d=Points(10000,-10000);
    OutPut(std::fixed,std::setprecision(3));
    std::vector<Lines>Arr;
    std::deque<Lines>Ans;
    Arr.push_back(Lines(b,a));
    Arr.push_back(Lines(d,c));
    getHpi(Arr,Ans);
    OutPut(Ans.size());
}
int main(){
    #ifdef LOCAL
    freopen("In.txt","r",stdin);
    freopen("Out.txt","w",stdout);
//  freopen("Err.txt","w",stderr);
    auto beg=std::chrono::steady_clock().now();
    #endif

    std::ios::sync_with_stdio(false);
```

```
    std::cin.tie(nullptr);std::cout.tie(nullptr);
    int Task=1;
    for(int Case=1;Case<=Task;Case++){
        Main(Case);
    }

    #ifdef LOCAL
    auto end=std::chrono::steady_clock().now();
    ErrPut(std::fixed,std::chrono::duration_cast<std::chrono::duration<double>>(end - beg).count(),'\n');
    #endif
    return 0;
}
```